Block Table

목적

Block Table은 **블록 헤더의 필드를 저장**하는 테이블입니다. 이 테이블은 EVM Circuit에서 블록 정보를 조회하는 데 사용되며, BLOCKHASH, COINBASE, TIMESTAMP, NUMBER, DIFFICULTY, GASLIMIT, BASEFEE, CHAINID 등의 opcode 실행 시 참조됩니다. 멀티 블록 증명에서는 여러 블록의 정보를 저장합니다.

테이블 스키마

파일: `zkevm-circuits/src/table.rs` (line 1349)

```
pub struct BlockTable {
/// 필드 태그 (BlockContextFieldTag)
pub tag: Column<Fixed>,
/// 인덱스 (블록 번호 또는 히스토리 인덱스)
pub index: Column<Advice>,
/// 필드 값
pub value: Column<Advice>,
}
```

컬럼 설명

1. tag (Fixed)

- 블록 필드를 식별하는 태그 (`BlockContextFieldTag`)
- Fixed 컬럼으로 할당되어 모든 블록에 대해 동일한 순서 유지
- 각 태그에 따라 `value` 컬럼의 의미가 달라짐

2. index (Advice)

- **일반 필드**: 블록 번호 (멀티 블록 증명용)
- **BlockHash**: 히스토리 인덱스 (최근 256개 블록)
- **다른 필드**: 블록 번호

3. value (Advice, SecondPhase)

- 필드의 실제 값
- RLC가 필요한 경우 SecondPhase에 할당

BlockContextFieldTag 열거형

파일: `zkevm-circuits/src/table.rs` (line 1295)

```
pub  enum BlockContextFieldTag {
Null = 0, // 사용되지 않음
Coinbase = 1, // 블록 채굴자 주소
Timestamp, // 블록 타임스탬프
Number, // 블록 번호
Difficulty, // 블록 난이도 (PoW) / Prevrandao (PoS)
GasLimit, // 블록 가스 한도
BaseFee = 8, // 기본 수수료 (EIP-1559)
BlockHash, // 이전 블록 해시
```

```
ChainId, // 체인 ID

NumTxs, // 블록 내 트랜잭션 개수

CumNumTxs, // 누적 트랜잭션 개수 (멀티 블록)

NumAllTxs, // 스킵된 L1 메시지 포함 총 트랜잭션 개수

}
```

필드 설명

1. Coinbase (tag = 1)

- **의미**: 블록 채굴자(또는 검증자) 주소
- **타입**: 20바이트 주소 (RLC 인코딩)
- **Opcode**: `COINBASE` (0x41)
- **예시**: `0x1234567890123456789012345678901234567890`

2. Timestamp (tag = 2)

- **의미**: 블록 생성 시간 (Unix timestamp)
- **타입**: uint64
- **Opcode**: `TIMESTAMP` (0x42)
- **예시**: `1609459200` (2021-01-01 00:00:00 UTC)

3. Number (tag = 3)

- **의미**: 블록 번호
- **타입**: uint64
- **Opcode**: `NUMBER` (0x43)

- **예시**: `12345678`

4. Difficulty (tag = 4)

- **의미**:
- **PoW**: 블록 난이도
- **PoS**: Prevrandao (EIP-4399)
- **타입**: uint256
- **Opcode**: `DIFFICULTY` (0x44) / `PREVRANDAO`
- **예시**: `2000000000000000` (PoW) 또는 랜덤 값 (PoS)

5. GasLimit (tag = 5)

- **의미**: 블록 가스 한도
- **타입**: uint64
- **Opcode**: `GASLIMIT` (0x45)
- **예시**: `30000000`

6. BaseFee (tag = 8)

- **의미**: 기본 수수료 (EIP-1559)
- **타입**: uint256
- **Opcode**: `BASEFEE` (0x48)
- **예시**: `20000000000` (20 Gwei)

7. BlockHash (tag = 9)

- **의미**: 이전 블록의 해시
- **타입**: 32바이트 해시 (RLC 인코딩)
- **Opcode**: `BLOCKHASH` (0x40)

- **특징**: 최근 256개 블록만 조회 가능
- **예시**: `0xabcd...`

8. ChainId (tag = 10)

- **의미**: 체인 ID
- **타입**: uint64
- **Opcode**: `CHAINID` (0x46)
- **예시**: `1` (Ethereum Mainnet), `534352` (Scroll Mainnet)

9. NumTxs (tag = 11)

- **의미**: 현재 블록에 포함된 트랜잭션 개수
- **타입**: uint64
- **사용**: 멀티 블록 증명에서 블록별 트랜잭션 개수 추적

10. CumNumTxs (tag = 12)

- **의미**: 누적 트랜잭션 개수 (현재 블록까지)
- **타입**: uint64
- **사용**: 멀티 블록 증명에서 트랜잭션 인덱싱

11. NumAllTxs (tag = 13)

- **의미**: 스킵된 L1 메시지를 포함한 총 트랜잭션 개수
- **타입**: uint64
- **사용**: Scroll의 L1-L2 메시지 처리

테이블 레이아웃

단일 블록

단일 블록 증명에서는 하나의 블록 정보만 저장됩니다:

```
tag | index | value

----------|-------|------------------

Null | 0 | 0

Coinbase | 12345 | 0x1234... (RLC)

Timestamp | 12345 | 1609459200

Number | 12345 | 12345

Difficulty| 12345 | 2000000000000000

GasLimit | 12345 | 30000000

BaseFee | 12345 | 20000000000

BlockHash | 0 | 0xabcd... (RLC)

BlockHash | 1 | 0xef01... (RLC)

...

BlockHash | 255 | 0x9876... (RLC)

ChainId | 12345 | 1

NumTxs | 12345 | 10

CumNumTxs | 12345 | 10

NumAllTxs | 12345 | 10
```

멀티 블록

멀티 블록 증명에서는 여러 블록의 정보를 저장합니다:

```
tag | index | value

----------|-------|------------------

Null | 0 | 0

Coinbase | 12345 | 0x1234... (블록 12345)

Coinbase | 12346 | 0x5678... (블록 12346)

Coinbase | 12347 | 0x9abc... (블록 12347)

Timestamp | 12345 | 1609459200

Timestamp | 12346 | 1609459212

Timestamp | 12347 | 1609459224

Number | 12345 | 12345

Number | 12346 | 12346

Number | 12347 | 12347

...

NumTxs | 12345 | 10 (블록 12345의 트랜잭션 개수)

NumTxs | 12346 | 15 (블록 12346의 트랜잭션 개수)

NumTxs | 12347 | 8 (블록 12347의 트랜잭션 개수)

CumNumTxs | 12345 | 10 (누적: 10)

CumNumTxs | 12346 | 25 (누적: 10 + 15)

CumNumTxs | 12347 | 33 (누적: 10 + 15 + 8)
```

BlockHash 히스토리

`BLOCKHASH` opcode는 최근 256개 블록의 해시를 조회할 수 있습니다:

```
tag | index | value

----------|-------|------------------

BlockHash | 0 | hash(block_number - 1)

BlockHash | 1 | hash(block_number - 2)

BlockHash | 2 | hash(block_number - 3)

...

BlockHash | 255 | hash(block_number - 256)
```

index는 상대적 오프셋을 나타냅니다:

- `index = 0`: 직전 블록
- `index = 1`: 2개 전 블록
- `index = 255`: 256개 전 블록

EVM Circuit과의 연동

EVM Circuit은 Block Table을 조회하여 블록 정보를 가져옵니다.

Opcode별 Lookup

COINBASE (0x41)

```
// 블록 채굴자 주소 조회

meta.lookup_any("coinbase", |meta| {

vec![

BlockContextFieldTag::Coinbase.expr(), // tag
```

```
block_number.expr(), // index
coinbase_address.expr(), // value
]
});

// 스택에 푸시
stack.push(coinbase_address);
```

TIMESTAMP (0x42)

```
// 블록 타임스탬프 조회
meta.lookup_any("timestamp", |meta| {
vec![
BlockContextFieldTag::Timestamp.expr(), // tag
block_number.expr(), // index
timestamp.expr(), // value
]
});

stack.push(timestamp);
```

NUMBER (0x43)

```
// 블록 번호 조회
```

```
meta.lookup_any("number", |meta| {
vec![
BlockContextFieldTag::Number.expr(), // tag
block_number.expr(), // index
block_number.expr(), // value (자기 자신)
]
});

stack.push(block_number);
```

DIFFICULTY (0x44) / PREVRANDAO

```
// 블록 난이도 또는 Prevrandao 조회
meta.lookup_any("difficulty", |meta| {
vec![
BlockContextFieldTag::Difficulty.expr(), // tag
block_number.expr(), // index
difficulty.expr(), // value
]
});

stack.push(difficulty);
```

GASLIMIT (0x45)

```
// 블록 가스 한도 조회
meta.lookup_any("gaslimit", |meta| {
vec![
BlockContextFieldTag::GasLimit.expr(), // tag
block_number.expr(), // index
gas_limit.expr(), // value
]
});

stack.push(gas_limit);
```

CHAINID (0x46)

```
// 체인 ID 조회
meta.lookup_any("chainid", |meta| {
vec![
BlockContextFieldTag::ChainId.expr(), // tag
block_number.expr(), // index
chain_id.expr(), // value
]
});
```

```
stack.push(chain_id);
```

BASEFEE (0x48)

```
// 기본 수수료 조회 (EIP-1559)
meta.lookup_any("basefee", |meta| {
vec![
BlockContextFieldTag::BaseFee.expr(), // tag
block_number.expr(), // index
base_fee.expr(), // value
]
});

stack.push(base_fee);
```

BLOCKHASH (0x40)

```
// 이전 블록 해시 조회
let  block_number_to_query = stack.pop();

// 범위 검증: block_number - 256 <= query < block_number
let  is_valid = (block_number - 256 <= block_number_to_query)
&& (block_number_to_query < block_number);
```

```
let history_index = block_number - block_number_to_query - 1;


meta.lookup_any("blockhash", |meta| {
vec![
BlockContextFieldTag::BlockHash.expr(), // tag
history_index.expr(), // index (0-255)
block_hash.expr(), // value
]
});


// 범위 밖이면 0 반환
let result = if is_valid { block_hash } else { 0 };
stack.push(result);
```

테이블 할당

Block Table은 `dev_load` 메서드를 통해 할당됩니다.

파일: `zkevm-circuits/src/table.rs` (line 1377)

```
impl BlockTable {
pub fn dev_load<F: Field>(
&self,
layouter: &mut impl Layouter<F>,
block_ctxs: &BlockContexts,
```

```
txs: &[Transaction],
challenges: &Challenges<Value<F>>,
) -> Result<(), Error> {
layouter.assign_region(
|| "block table",
|mut region| {
let mut offset = 0;
// All-zero 행 할당
for column in block_table_columns.iter() {
region.assign_advice(
|| "block table all-zero row",
*column,
offset,
|| Value::known(F::zero()),
)?;
}
offset += 1;


// 각 블록 컨텍스트에 대해
let mut cum_num_txs = 0usize;
for block_ctx in block_ctxs.ctxs.values() {
let num_txs = txs
.iter()
```

```
.filter(|tx| tx.block_number == block_ctx.number.as_u64())
.count();
cum_num_txs += num_txs;
// 블록 필드 할당
for row in block_ctx.table_assignments(
num_txs,
cum_num_txs,
0,
challenges
) {
// tag (Fixed), index, value 할당
offset += 1;
}
}

Ok(())
},
)
}
}
```

BlockContext 구조체

파일: zkevm-circuits/src/witness/block.rs

```rust
pub struct BlockContext {
pub coinbase: Address,
pub timestamp: Word,
pub number: Word,
pub difficulty: Word,
pub gas_limit: u64,
pub base_fee: Word,
pub history_hashes: Vec<Word>, // 최근 256개 블록 해시
pub chain_id: u64,
}

impl BlockContext {
pub fn table_assignments<F: Field>(
&self,
num_txs: usize,
cum_num_txs: usize,
num_all_txs: usize,
challenges: &Challenges<Value<F>>,
) -> Vec<[Value<F>; 3]> {
// [tag, index, value] 형식의 행 생성
}
```

```
}
```

멀티 블록 증명

멀티 블록 증명에서는 여러 블록의 정보를 하나의 Block Table에 저장합니다.

BlockContexts 구조체

```rust
pub struct BlockContexts {
pub ctxs: BTreeMap<u64, BlockContext>, // block_number -> BlockContext
}
```

누적 트랜잭션 개수

`CumNumTxs` 는 청크 내 트랜잭션 인덱싱에 사용됩니다:

```
블록 12345: NumTxs = 10, CumNumTxs = 10
블록 12346: NumTxs = 15, CumNumTxs = 25
블록 12347: NumTxs = 8, CumNumTxs = 33

트랜잭션 인덱스:
- 블록 12345의 트랜잭션: 1-10
- 블록 12346의 트랜잭션: 11-25
- 블록 12347의 트랜잭션: 26-33
```

성능 최적화

1. Fixed 컬럼 사용

`tag` 컬럼은 Fixed로 할당되어:

- 모든 블록에 대해 동일한 순서 유지
- 제약 조건 간소화
- 회로 크기 감소

2. SecondPhase 컬럼

RLC를 사용하는 값들은 SecondPhase에 할당:

- `value` : 주소, 해시 등 RLC 인코딩 값

3. 히스토리 해시 최적화

최근 256개 블록 해시만 저장하여 테이블 크기 제한

제약 조건

1. 블록 번호 일관성

```
// Number 필드의 value는 index와 동일해야 함
value == index (when  tag == Number)
```

2. BLOCKHASH 범위 검증

```
// BLOCKHASH는 최근 256개 블록만 조회 가능
```

```
// block_number - 256 <= query < block_number
```

3. 누적 트랜잭션 개수

```
// CumNumTxs는 단조 증가

CumNumTxs[i] = CumNumTxs[i-1] + NumTxs[i]
```

예시: 블록 정보 조회

Solidity 코드:

```
function getBlockInfo() public view returns (

address coinbase,

uint timestamp,

uint number,

uint difficulty,

uint gasLimit,

uint baseFee,

uint chainId

) {

coinbase = block.coinbase;

timestamp = block.timestamp;

number = block.number;
```

```
difficulty = block.difficulty;

gasLimit = block.gaslimit;

baseFee = block.basefee;

chainId = block.chainid;

}
```

Block Table 조회:

```
Opcode | Tag | Index | Value

-----------|-----------|-------|------------------

COINBASE | Coinbase | 12345 | 0x1234...

TIMESTAMP | Timestamp | 12345 | 1609459200

NUMBER | Number | 12345 | 12345

DIFFICULTY | Difficulty| 12345 | 2000000000000000

GASLIMIT | GasLimit | 12345 | 30000000

BASEFEE | BaseFee | 12345 | 20000000000

CHAINID | ChainId | 12345 | 1
```

디버깅 팁

블록 정보 덤프

```
// 블록 정보 출력

for (block_number, block_ctx) in  block_ctxs.ctxs.iter() {
```

```
println!("Block {}: coinbase={:?}, timestamp={}, gas_limit={}",
block_number, block_ctx.coinbase, block_ctx.timestamp, block_ctx.gas_limit);
}
```

일반적인 오류

1. **블록 번호 불일치**: index와 value가 맞지 않음 (Number 필드)
2. **BLOCKHASH 범위 오류**: 256개 이상 이전 블록 조회 시도
3. **누적 트랜잭션 개수 오류**: CumNumTxs 계산 오류

관련 문서

- EVM Circuit - Block Table을 조회하는 회로
- Tx Table - 트랜잭션 정보 테이블
- RLC 기법 - RLC 인코딩 상세

참고 자료

- **코드**: `zkevm-circuits/src/table.rs` (line 1349)
- **Witness**: `zkevm-circuits/src/witness/block.rs`
- **EIP-1559**: https://eips.ethereum.org/EIPS/eip-1559
- **EIP-4399**: https://eips.ethereum.org/EIPS/eip-4399 (Prevrandao)